

Esercizio 13 PC

giovedì 26 marzo 2026 14:37

TIMER e MOUSE sono due periferiche interfacciate direttamente al processore z64. TIMER lancia interruzioni ogni TOT nanosecondi verso lo z64. Il servizio associato all'interruzione è il seguente: il processore legge dalla periferica MOUSE le nuove coordinate X, Y (esprese ciascuna come word) e le memorizza sequenzialmente in un buffer globale in memoria formato da 128 longword. Ogni longword è pertanto costituita dalla coppia (XY). Ogni 100 letture lo z64 calcola il massimo, come longword, tra le coppie memorizzate e lo memorizza in un'altra variabile globale. Il puntatore per scrivere i successivi 128 valori viene quindi resettato, per ricominciare a scrivere dall'inizio del buffer. Si noti che il driver ad ogni lettura legge una sola coordinata (la prima volta la X e la seconda la Y). Il servizio di acquisizione e calcolo della media è ciclico e non prevede arresto. Progettare l'interfaccia di MOUSE e TIMER. La routine di inizializzazione ed il driver per la gestione delle interruzioni di TIMER.

Progettare:

- l'interfaccia di MOUSE
- l'interfaccia e lo SCA di TIMER
- il software di attivazione ed il driver di TIMER.

Soluzione

Si presuppone mouse bw e timer irq

Si presuppone come coppia xy la somma

```
.org 0x800
.data:
    .equ $mouse_status,0x0 #non c'è reg in quanto non ritenuto necessario
    .equ $timer_reg,0x1
    .equ $timer_irq,0x2
    Timer : .word 0
    Vettore : .fill 128,0,32
    Massimo : .long 0
    X: .word 0
    Y: .word 0
.text
Main:
    Movl $0,%ecx
    Movl $timer_reg,%al
    .loop:
        Addl $1,%ecx
        Cmpl %ecx,%al
        Jnz .loop
        Jz .attiva
    .attiva:
        Movl $1,$al
        Outb %al,$timer_irq #attivo interrupt
.driver0: #timer
    Popq %rax
    Popq %rcx
    Movl $0,$al
    Outb %al,$timer_irq #azzerò richiesta
    Call prendi
    Popq %rcx
    Popq %rax
    Iret
Prendi:
    Movl $0,%ecx
    .ciclo:
        Movl $0,%al
        Out,b %al,$mouse_status
    .bw:
        Inl $mouse_status,%al
        Btb $0,%al
        Jnc .bw
    #dati pronti
    Addl X,vettore(,%ecx,4) #aggiungo x al vettore
    Movl $0,%al
```

```

Out,b %al,$mouse_status
.bw1:
    Inl $mouse_status,%al
    Btb $0,%al
    Jnc .bw1
#dati pronti
Addl Y,vettore(,%ecx,4) #aggiungo x al vettore
Addl $1,%ecx
Cmpl $128,%ecx
Jnz .loop
Jz .massimo

```

Massimo:

```

Movl $0,%ecx
Movl %ecx,%r8d
Addl $1,%r8d
Movl $0,%r9d
Call f

```

F:

```

.loop:
    Cmpl vett(,%ecx,4),vett(,%r8d,4)
    Jnz .A_max
    Jz .B_max
    Addl $1,%r9d
    Cmpl $100,%r9d
    Jnz .loop
    Jz .riposiziona

```

.A_max:

```

Movl vettore(,%ecx,4),massimo
Jmp .inc

```

.B_max:

```

Movl vettore(,%r9d,4),massimo
Jmp.inc

```

.inc:

```

Addl $1,%ecx
Addl $1,%r8d
Jmp f

```

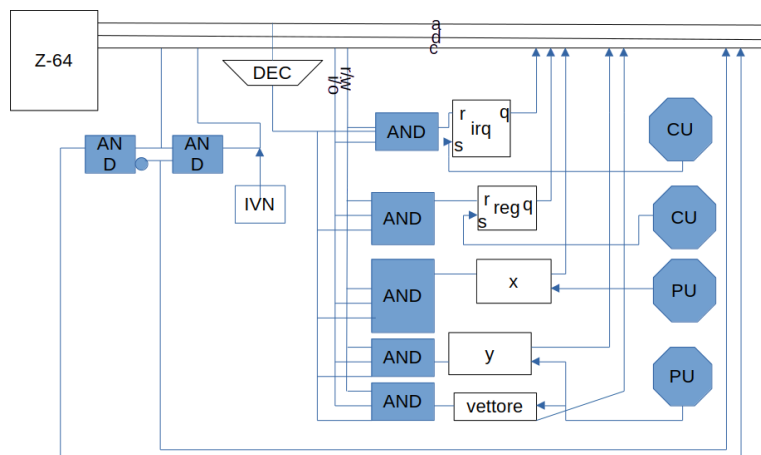
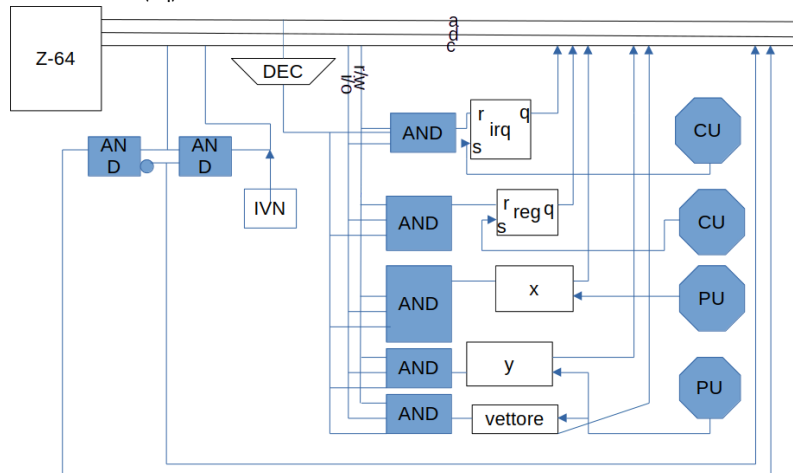
.riposiziona:

```

Movl $0,%ecx

```

Schema di timer (irq):



Schema di mouse (bw):

